

Architecture Production Code

Arquitectura Mejorada

Estructura de Capas

La arquitectura ha sido refactorizada siguiendo principios de diseño limpio y separación de responsabilidades:

```
production_code/ ■■■■ api/ # Capa de API (Presentación) ■ ■■■■ routes/ # Rutas
organizadas por dominio ■ ■ ■■■■ memory.py ■ ■ ■■■■ redundancy.py ■ ■ ■■■■ pipeline.py
■ ■ ■■■■ chat.py ■ ■ ■■■■ config.py ■ ■ ■■■■ monitoring.py ■ ■ ■■■■ health.py ■ ■■■■
dependencies.py # Inyección de dependencias ■ ■■■■ api_utils.py # Utilidades de
validación ■ ■■■■ middleware.py # Middleware personalizado ■■■■ services/ # Capa de
Servicios (Lógica de Negocio) ■ ■■■■ pipeline_service.py ■ ■■■■ memory_service.py ■
■■■■ redundancy_service.py ■ ■■■■ chat_service.py ■ ■■■■ config_service.py ■ ■■■■
monitoring_service.py ■■■■ core/ # Capa Core (Utilidades y Base) ■ ■■■■ paper_base.py ■
■■■■ utils.py ■ ■■■■ error_handling.py ■ ■■■■ ... ■■■■ application.py # Factory de
aplicación ■■■■ api_server.py # Punto de entrada
```

Principios de Diseño

1. Separación de Responsabilidades

- ****API Layer****: Maneja HTTP, validación de requests, y respuestas
- ****Service Layer****: Contiene la lógica de negocio
- ****Core Layer****: Utilidades compartidas y clases base

2. Inyección de Dependencias

Los servicios se inyectan a través de FastAPI's `Depends()`:

```
@router.post("/store") async def store_episode( request: MemoryStoreRequest, service:
MemoryService = Depends(get_memory_service) ): ...
```

3. Gestión del Ciclo de Vida

La aplicación usa `Lifespan` context manager para inicializar y limpiar recursos:

```
@asynccontextmanager async def lifespan(app: FastAPI): # Startup
    initialize_services(...) yield # Shutdown
```

4. Manejo de Errores

- Errores de validación: HTTP 400
- Errores de servicio: HTTP 500
- Servicios no disponibles: HTTP 503
- Middleware centralizado para logging y manejo de excepciones

Flujo de Datos

```
Request → API Route → Validation → Service → Pipeline/Module → Response
```

1. **Request:** Llega a la ruta API
2. **Validation:** `api_utils` valida y convierte datos
3. **Service:** Ejecuta lógica de negocio
4. **Pipeline/Module:** Accede a módulos de bajo nivel
5. **Response:** Retorna resultado formateado

Ventajas de la Nueva Arquitectura

1. **Testabilidad:** Servicios pueden ser testeados independientemente
2. **Mantenibilidad:** Código organizado por responsabilidades
3. **Escalabilidad:** Fácil agregar nuevos endpoints y servicios
4. **Reutilización:** Servicios pueden ser usados por CLI, API, u otros interfaces
5. **Separación de Concerns:** Cambios en una capa no afectan otras

Migración desde `api_unified.py`

El archivo `api_unified.py` sigue disponible para compatibilidad, pero se recomienda usar:

```
from application import create_app app = create_app()
```

Próximos Pasos

- [] Consolidar configuración (eliminar duplicación)
- [] Agregar tests para servicios
- [] Documentar APIs con OpenAPI mejorado

- ☐ Agregar rate limiting
- ☐ Implementar autenticación/autorización